

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Ордена трудового Красного Знамени федеральное государственное
бюджетное**

**образовательное учреждение высшего образования
«Московский технический университет связи и информатики»**

Кафедра “Программная инженерия”

Отчет по Лабораторной работе №5.

Выполнил: Студент группы БПИ2404

Сидельников Николай Станиславович

Проверил: Адедиха Коффи Жермен

Москва

2026

Цель работы

Изучить основы безопасности в Spring Boot-приложениях, научиться настраивать аутентификацию и авторизацию пользователей, использовать Spring Security, JWT и роли пользователей.

Теоретические сведения

Spring Security - основной механизм обеспечения безопасности в экосистеме Spring.

Основные	задачи	Spring	Security:
-	аутентификация		пользователя;
-	авторизация		доступа;
-	защита		URL;
-	работа	с	ролями;
-	хранение информации	о	пользователе.

В лабораторной работе реализованы два подхода:

1. HTTP Basic Authentication.
2. JWT (JSON Web Token).

JWT позволяет организовать stateless-аутентификацию, когда сервер не хранит состояние пользователя.

Клиент хранит токен и отправляет его в каждом запросе.

Структура проекта

Пакет	Назначение
config	Конфигурация Spring Security.
controller	REST-контроллеры приложения.

service	Бизнес-логика приложения.
repository	Работа с базой данных через JPA.
security	JWT, UserDetails и фильтры безопасности.
model/entity	Сущности базы данных.
model/dto	DTO для запросов и ответов API.

Разбор SecurityConfig

```

package org.example.config;

import lombok.RequiredArgsConstructor;
import org.example.security.CustomUserDetailsService;
import org.example.security.JwtAuthenticationFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationF

```

ilter;

@Configuration

@EnableMethodSecurity

@RequiredArgsConstructor

```
public class SecurityConfig {
```

```
    private final CustomUserDetailsService customUserDetailsService;
```

```
    private final JwtAuthenticationFilter jwtAuthenticationFilter;
```

```
                                @Bean
```

```
                                public PasswordEncoder passwordEncoder() {  
                                    return new BCryptPasswordEncoder();  
                                }
```

```
                                @Bean
```

```
                                public DaoAuthenticationProvider authenticationProvider() {  
                                    DaoAuthenticationProvider provider = new  
DaoAuthenticationProvider(customUserDetailsService);  
                                    provider.setPasswordEncoder(passwordEncoder());  
                                    return provider;  
                                }
```

```
                                @Bean
```

```
                                public AuthenticationManager authenticationManager(AuthenticationConfiguration  
configuration) throws Exception {  
                                    return configuration.getAuthenticationManager();  
                                }
```

@Bean

```
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception
{
    http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/auth/**", "/hello").permitAll()
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .requestMatchers("/users/**").hasAnyRole("USER", "ADMIN")
            .requestMatchers("/notifications/**").hasAnyRole("USER",
"ADMIN")
            .anyRequest().authenticated()
        )
        .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authenticationProvider(authenticationProvider())
        .addFilterBefore(jwtAuthenticationFilter,
UsernamePasswordAuthenticationFilter.class)
        .formLogin(form -> form.disable())
        .httpBasic(httpBasic -> {});

    return http.build();
}
}
```

SecurityConfig является главным конфигурационным классом безопасности.

В конфигурации:
- CSRF;
- настроены роли пользователей;
- включён HTTP Basic;
- включён JWT-фильтр;
- используется BCryptPasswordEncoder;
- приложение работает в режиме STATELESS.

Разбор сущности User

```
package org.example.model.entity;  
  
import jakarta.persistence.*;  
import lombok.Getter;  
import lombok.NoArgsConstructor;  
import lombok.Setter;  
import org.example.model.enums.UserRole;  
  
import java.time.LocalDateTime;  
import java.util.ArrayList;  
import java.util.List;  
  
@Getter  
@Setter  
@NoArgsConstructor
```

```

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(nullable = false)
    private String password;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private UserRole role;

    private String phone;
    private String deviceToken;
    private String telegramChatId;
    private LocalDateTime createdAt;

    @OneToMany(mappedBy = "recipient", cascade = CascadeType.ALL)
    private List<Notification> notifications = new ArrayList<>();

```

}

Сущность User хранит:

- имя пользователя;
- email;
- пароль;
- роль;
- данные для уведомлений;
- дату регистрации.

Email используется как логин пользователя.

Пароль хранится в зашифрованном виде.

Роли пользователей

```
package org.example.model.enums;
```

```
public enum UserRole {
```

```
    ROLE_USER,
```

```
    ROLE_ADMIN
```

```
}
```

В приложении реализованы две роли:

- ROLE_USER - обычный пользователь;
- ROLE_ADMIN - администратор.

Роли используются для ограничения доступа к URL.

Разбор CustomUserDetails

```
package org.example.security;

import lombok.AllArgsConstructor;
import org.example.model.entity.User;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

import java.util.Collection;
import java.util.List;

@AllArgsConstructor
public class CustomUserDetails implements UserDetails {

    private final User user;

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return List.of(new SimpleGrantedAuthority(user.getRole().name()));
    }

    @Override
    public String getPassword() {
        return user.getPassword();
    }
}
```

```
}
```

```
@Override
```

```
public String getUsername() {  
    return user.getEmail();  
}
```

```
@Override
```

```
public boolean isAccountNonExpired() {  
    return true;  
}
```

```
@Override
```

```
public boolean isAccountNonLocked() {  
    return true;  
}
```

```
@Override
```

```
public boolean isCredentialsNonExpired() {  
    return true;  
}
```

```
@Override
```

```
public boolean isEnabled() {  
    return true;  
}
```

```
public User getUser() {
```

```

        return user;
    }
}

```

CustomUserDetails является адаптером между сущностью User и Spring Security.

Класс:

```

-      предоставляет      email      как      username;
-      предоставляет      пароль;
-      преобразует      роль      пользователя      в      GrantedAuthority.

```

Разбор CustomDetailsService

```

package                                org.example.security;

import                                lombok.RequiredArgsConstructor;
import                                org.example.model.entity.User;
import                                org.example.repository.UserRepository;
import                                org.springframework.security.core.userdetails.UserDetails;
import                                org.springframework.security.core.userdetails.UserDetailsService;
import                                org.springframework.security.core.userdetails.UsernameNotFoundException;
import                                org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
public class CustomDetailsService implements UserDetailsService {

```

```

        private final UserRepository userRepository;

        @Override
        public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
            User user = userRepository.findByEmail(username)
                .orElseThrow(() -> new UsernameNotFoundException("Пользователь не
найден"));

            return new CustomUserDetails(user);
        }
}

```

CustomUserDetailsService загружает пользователя из базы данных.
Поиск выполняется по email.
Если пользователь не найден - выбрасывается UsernameNotFoundException.

Разбор AuthService

```

package org.example.service;

import lombok.RequiredArgsConstructor;
import org.example.model.dto.RegisterRequest;
import org.example.model.entity.User;
import org.example.model.enums.UserRole;
import org.example.repository.UserRepository;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

```

```

import                                     java.time.LocalDateTime;

@Service
@RequiredArgsConstructor
public class AuthService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public void registerUser(RegisterRequest request) {
        register(request, UserRole.ROLE_USER);
    }

    public void registerAdmin(RegisterRequest request) {
        register(request, UserRole.ROLE_ADMIN);
    }

    private void register(RegisterRequest request, UserRole role) {
        if (userRepository.existsByEmail(request.getEmail())) {
            throw new IllegalArgumentException("Пользователь с таким email уже
существует");
        }

        User user = new User();
        user.setName(request.getName());
        user.setEmail(request.getEmail());
        user.setPassword(passwordEncoder.encode(request.getPassword()));
    }
}

```

```

        user.setRole(role);
        user.setCreatedAt(LocalDateTime.now());
        userRepository.save(user);
    }
}

```

AuthService отвечает за регистрацию пользователя.

При регистрации:

- создаётся новый объект User;
- пароль шифруется через BCrypt;
- пользователю назначается ROLE_USER;
- объект сохраняется в PostgreSQL.

Разбор AuthController

```

package org.example.controller;

import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.example.model.dto.AuthResponse;
import org.example.model.dto.LoginRequest;
import org.example.model.dto.RegisterRequest;
import org.example.security.JwtService;
import org.example.service.AuthService;
import org.springframework.security.authentication.AuthenticationManager;
import

```

```
org.springframework.security.authentication.UsernamePasswordAuthenticationToken
;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
```

```
@RequestMapping("/auth")
```

```
@RequiredArgsConstructor
```

```
public class AuthController {
```

```
    private final AuthService authService;
```

```
    private final AuthenticationManager authenticationManager;
```

```
    private final JwtService jwtService;
```

```
    @PostMapping("/register")
```

```
    public String register(@RequestBody @Valid RegisterRequest request) {
```

```
        authService.registerUser(request);
```

```
        return "Пользователь успешно зарегистрирован";
```

```
    }
```

```
    @PostMapping("/register-admin")
```

```
    public String registerAdmin(@RequestBody @Valid RegisterRequest request) {
```

```
        authService.registerAdmin(request);
```

```
        return "Администратор успешно зарегистрирован";
```

```
    }
```

```

// Удобный вариант для Postman: возвращает JSON { "token": "..." }.
                                @PostMapping("/login")
public AuthResponse login(@RequestBody @Valid LoginRequest request) {
                                authenticationManager.authenticate(
                                new UsernamePasswordAuthenticationToken(request.getEmail(),
request.getPassword())
                                );
    return new AuthResponse(jwtService.generateToken(request.getEmail()));
}

// Упрощенный вариант: возвращает только строку токена без JSON-обертки.
                                @PostMapping("/login-raw")
public String loginRaw(@RequestBody @Valid LoginRequest request) {
                                authenticationManager.authenticate(
                                new UsernamePasswordAuthenticationToken(request.getEmail(),
request.getPassword())
                                );
    return jwtService.generateToken(request.getEmail());
}
}

```

AuthController

реализует:

- регистрацию пользователя;
- JWT-логин;
- выдачу токена после успешной аутентификации.

Разбор JwtService

```
package org.example.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;

@Service
public class JwtService {

    private static final String SECRET =
"verySecretKeyForJwtTokenVerySecretKey12345";

    private static final long EXPIRATION_MS = 1000 * 60 * 60;

    public String generateToken(String username) {
        return Jwts.builder()
            .subject(username)
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + EXPIRATION_MS))
            .signWith(getSigningKey())
            .compact();
    }
}
```

```

        public String extractUsername(String token) {
            return extractAllClaims(token).getSubject();
        }

        public boolean isTokenValid(String token, UserDetails userDetails) {
            String username = extractUsername(token);
            return username.equals(userDetails.getUsername()) && !isTokenExpired(token);
        }

        public boolean isTokenValid(String token) {
            return !isTokenExpired(token);
        }

        private boolean isTokenExpired(String token) {
            return extractAllClaims(token).getExpiration().before(new Date());
        }

        private Claims extractAllClaims(String token) {
            return Jwts.parser()
                .verifyWith(getSigningKey())
                .build()
                .parseSignedClaims(token)
                .getPayload();
        }

        private SecretKey getSigningKey() {
            return Keys.hmacShaKeyFor(SECRET.getBytes(StandardCharsets.UTF_8));
        }

```

```

    }
}

```

```

JwtService                                отвечает за:
-                                           создание JWT-токена;
-           чтение username из токена;
-           проверку срока действия;
-           проверку валидности токена.

```

Токен подписывается с использованием секретного ключа и алгоритма HS256.

Разбор JwtAuthenticationFilter

```

package                                     org.example.security;

import                                     jakarta.servlet.FilterChain;
import                                     jakarta.servlet.ServletException;
import                                     jakarta.servlet.http.HttpServletRequest;
import                                     jakarta.servlet.http.HttpServletResponse;
import                                     lombok.RequiredArgsConstructor;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken
;
import                                     org.springframework.security.core.context.SecurityContextHolder;
import                                     org.springframework.security.core.userdetails.UserDetails;
import                                     org.springframework.security.core.userdetails.UserDetailsService;
import

```

```

org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import                                     org.springframework.stereotype.Component;
import                                     org.springframework.web.filter.OncePerRequestFilter;

import                                     java.io.IOException;

@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                     HttpServletResponse response,
                                     FilterChain filterChain) throws ServletException, IOException
    {
        String authHeader = request.getHeader("Authorization");

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        String token = authHeader.substring(7);

        try {

```

```

        String username = jwtService.extractUsername(token);

        if (username != null &&
            SecurityContextHolder.getContext().getAuthentication() == null) {

            UserDetails userDetails =
userDetailsService.loadUserByUsername(username);

            if (jwtService.isTokenValid(token, userDetails)) {
                UsernamePasswordAuthenticationToken authenticationToken =
                    new UsernamePasswordAuthenticationToken(
                        userDetails,
                        null,
                        userDetails.getAuthorities()
                    );
                authenticationToken.setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));
                SecurityContextHolder.getContext().setAuthentication(authenticationTok
en);
            }
        }
    } catch (Exception ex) {
        // Если токен плохой или просрочен, просто не аутентифицируем запрос.
        // Далее Spring Security сам вернет 401/403 для защищенного URL.
        SecurityContextHolder.clearContext();
    }

    filterChain.doFilter(request, response);
}

```

}

JWT-фильтр:

- читает Authorization Bearer Token;
- извлекает пользователя из JWT;
- загружает пользователя через UserDetailsService;
- помещает Authentication в SecurityContext.

Зависимости проекта

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.0.5</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>org.example</groupId>
    <artifactId>spring-lab3-notifications</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>spring-lab3-notifications</name>
    <description>spring-lab3-notifications</description>
```

```
        <url/>
    </licenses>
    <license/>
</licenses>
<developers>
    <developer/>
</developers>
    <scm>
    <connection/>
    <developerConnection/>
    <tag/>
    <url/>
    </scm>
    <properties>
    <java.version>17</java.version>
    </properties>
    <dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>
```

```
        <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc-test</artifactId>
        <scope>test</scope>
    </dependency>
```

```
        <!--      JPA      -->
        <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
```

```
        <!--      PostgreSQL      -->
        <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <scope>runtime</scope>
    </dependency>
```

```
        <!--      Lombok      -->
        <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
```

```
        <!--      Validation      -->
        <dependency>
```



```
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
```

```
        <!--      Spring      Security      -->
        <dependency>
```

```
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
```

```
        <!--      JWT      -->
        <dependency>
```

```
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.12.6</version>
    </dependency>
```

```
        <dependency>
```

```
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-impl</artifactId>
        <version>0.12.6</version>
        <scope>runtime</scope>
    </dependency>
```

```
        <dependency>
```

```
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-jackson</artifactId>
        <version>0.12.6</version>
        <scope>runtime</scope>
    </dependency>
```

```

</dependencies>

</build>
</plugins>
</plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>

```

ОСНОВНЫЕ	ЗАВИСИМОСТИ:			
-	Spring	Boot	Starter	Web;
-		Spring		Security;
-	Spring		Data	JPA;
-		PostgreSQL		Driver;
-	JWT	(jjwt-api,	jjwt-impl,	jjwt-jackson);
-				Lombok;
-		Validation		API.

Проверка работы через Postman

- POST /auth/register - регистрация пользователя.
- POST /auth/login - получение JWT-токена.

- GET /users/all - защищённый endpoint.
- GET /notifications/all - защищённый endpoint.
- GET /admin/ping - endpoint только для администратора.

Запуск проекта

Для запуска проекта использовались команды:

```
mvn clean install
mvn spring-boot:run
```

Приложение запускается на порту 8080.
PostgreSQL используется как основная база данных.

Ответы на контрольные вопросы

Что такое аутентификация? Проверка личности пользователя.

Что такое авторизация? Проверка прав доступа пользователя.

Что делает SecurityFilterChain? Настраивает правила безопасности приложения.

Для чего нужен PasswordEncoder? Для шифрования и проверки паролей.

Что такое JWT? Токен для stateless-аутентификации.

Что хранится в SecurityContext? Информация о текущем пользователе.

Что делает JwtAuthenticationFilter? Проверяет JWT и авторизует пользователя.

Чем JWT отличается от сессии? JWT не хранит состояние на сервере.

Вывод

В ходе лабораторной работы была реализована система безопасности для Spring Boot-приложения.

Были		изучены:
-	Spring	Security;
-	аутентификация	и авторизация;
-		JWT;
-	HTTP	Basic;
-	роли	пользователей;
-	фильтры	безопасности;
-		BCrypt;
-		SecurityContext.

Проект успешно поддерживает регистрацию, логин, защиту URL и авторизацию по ролям.

Дополнительные разделы по методическим указаниям

HTTP Basic и JWT

HTTP Basic передает логин и пароль в каждом запросе. JWT использует токен, который клиент сохраняет после логина. JWT лучше подходит для REST API и мобильных приложений.

BCrypt PasswordEncoder

BCrypt используется для безопасного хранения паролей. Пароль никогда не хранится в открытом виде.

SecurityFilterChain

SecurityFilterChain определяет правила безопасности, доступ к URL и порядок фильтров Spring Security.

Роли пользователей

ROLE_USER и ROLE_ADMIN используются для ограничения доступа к различным URL приложения.

JWT Authentication Filter

JWT-фильтр проверяет Authorization Bearer Token и создает Authentication внутри SecurityContext.

STATELESS режим

При использовании JWT сервер не хранит состояние пользователя между запросами.

Дополнительные ответы на контрольные вопросы

Что такое Authentication? Процесс проверки личности пользователя.

Что такое Authorization? Проверка прав доступа пользователя.

Что делает PasswordEncoder? Шифрует и проверяет пароль.

Что хранится в JWT? Информация о пользователе и срок действия токена.

Что делает SecurityContext? Хранит текущего аутентифицированного пользователя.

Что такое Bearer Token? JWT-токен, передаваемый в Authorization header.

Почему JWT называют stateless? Сервер не хранит сессию пользователя.

Что делает @EnableMethodSecurity? Позволяет защищать методы аннотациями.